
Esthevão Marttioly

Problem Set III

Question 1

We want to minimize the function:

$$f(x) = x \cdot \sin(5x), \quad x \in [0, 10]$$

We implemented three different optimization methods: a grid search (heuristic), a derivative-based method (Newton-Raphson), and a stochastic global method (CRS).

The stopping criterion is based on tolerances for the objective function and/or a maximum number of evaluations. In our case, we used relative and absolute tolerances for the function value, along with a cap on the number of iterations.

Grid Search The grid search evaluates the function over a discretized version of the interval $[0, 10]$ and selects the point that minimizes $f(x)$. This method does not rely on derivatives and is guaranteed to find the global minimum *within the grid*. In this case, this method is great, because we have a specific interval with just one dimension.

Since grid search is not iterative in the same sense as the other methods, tolerance criteria are not directly applicable. Instead, precision is controlled by the grid resolution.

Newton-Raphson The Newton-Raphson method is a derivative-based algorithm that finds critical points by solving $f'(x) = 0$. The update rule is:

$$x_{n+1} = x_n - \frac{f'(x_n)}{f''(x_n)}$$

This method uses both first and second derivatives and typically converges very quickly (quadratically) when starting close to a solution. However, it is very sensible to local minima, so well as to x_0 , because it doesn't evaluate the founded root and just find some critical point.

CRS (Controlled Random Search) The CRS algorithm is a stochastic global optimization method. It starts with a population of random points in the domain and iteratively replaces worse solutions with better randomly generated candidates.

This method does not use derivatives and is designed to explore the parameter space broadly, reducing the risk of being trapped in local minima.

Results

The results obtained are summarized in Table 1.

Table 1: Optimization Methods for $f(x)$

	Grid Search	Newton	CRS
Solution	9.74297429742974	-5.97571732050998	9.68515580461575
Objective	-9.74098948412058	-5.97237323183733	-9.33708902405925
Iterations	1	5	101

Grid search finds a point close to $x \approx 9.74$, which corresponds to a very low function value. CRS also finds a point in the same region, suggesting that this is close to the global minimum.

Newton-Raphson, when initialized at $x_0 = 0$, converges to $x \approx -5.98$, which is outside the interval of interest and corresponds to a different critical point.

However, when initialized at $x_0 = 9$, Newton-Raphson converges to: $x^* \approx 9.115$, in only 3 iterations, showing that it is largely depend on initial values, since this function is cyclical.

The results also depend strongly on the choice of algorithm:

- **Grid Search:** robust and simple, but accuracy depends on grid resolution and computational cost.
- **Newton-Raphson:** very fast and efficient, but local in nature and highly sensitive to the starting point.
- **CRS:** explores the parameter space globally and is less sensitive to initial values, but requires more iterations and is computationally more expensive.

Question 2

We are given the function:

$$f(x_1, x_2; \theta) = \theta_1 x_1 + \theta_2 e^{-x_2^2} + \theta_3 \log(1 + |x_2|) + \theta_4 x_1^{x_2}$$

and four observations:

$$f(1, 1; \theta_0) = y_1 = 43.614$$

$$f(2, 4; \theta_0) = y_2 = 563.694$$

$$f(-1, 2; \theta_0) = y_3 = 43.230$$

$$f(2, -2; \theta_0) = y_4 = 23.130$$

We define the loss function as the sum of squared differences between model predictions and observed values:

$$g(\theta) = \sum_{i=1}^4 (f(x_{1i}, x_{2i}; \theta) - y_i)^2$$

This corresponds to a nonlinear least squares problem.

Evaluating the loss function at the initial guess $\theta = (0, 0, 0, 0)$, we obtain a large value of $g(0) = 322,056.9$, that reflects a poor initial fit of the model.

(c) Optimization Algorithm

To minimize $g(\theta)$, we used the Simulated Annealing (SANN) algorithm.

Simulated Annealing is a stochastic global optimization method inspired by thermodynamics. It allows occasional uphill moves, which helps avoid getting trapped in local minima. The algorithm starts with a high “temperature” to promote exploration and gradually reduces it to focus on exploitation.

We initialized the algorithm at $\theta^{(0)} = (0, 0, 0, 0)$ and set a sufficiently large number of iterations to ensure convergence.

Figure 1 shows the evolution of the loss function over iterations and figure 2 presents the evolution of each parameter θ_j over iterations. The parameters initially fluctuate significantly due to exploration, and later stabilize as the algorithm converges.

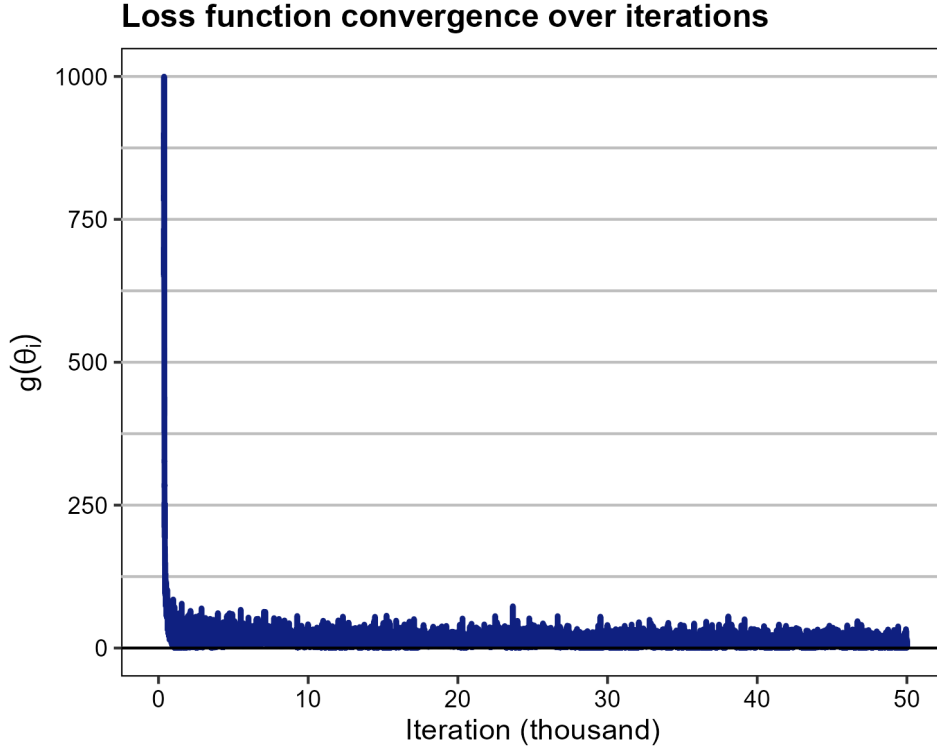


Figure 1: Convergence of the loss function $g(\theta)$ over iterations.

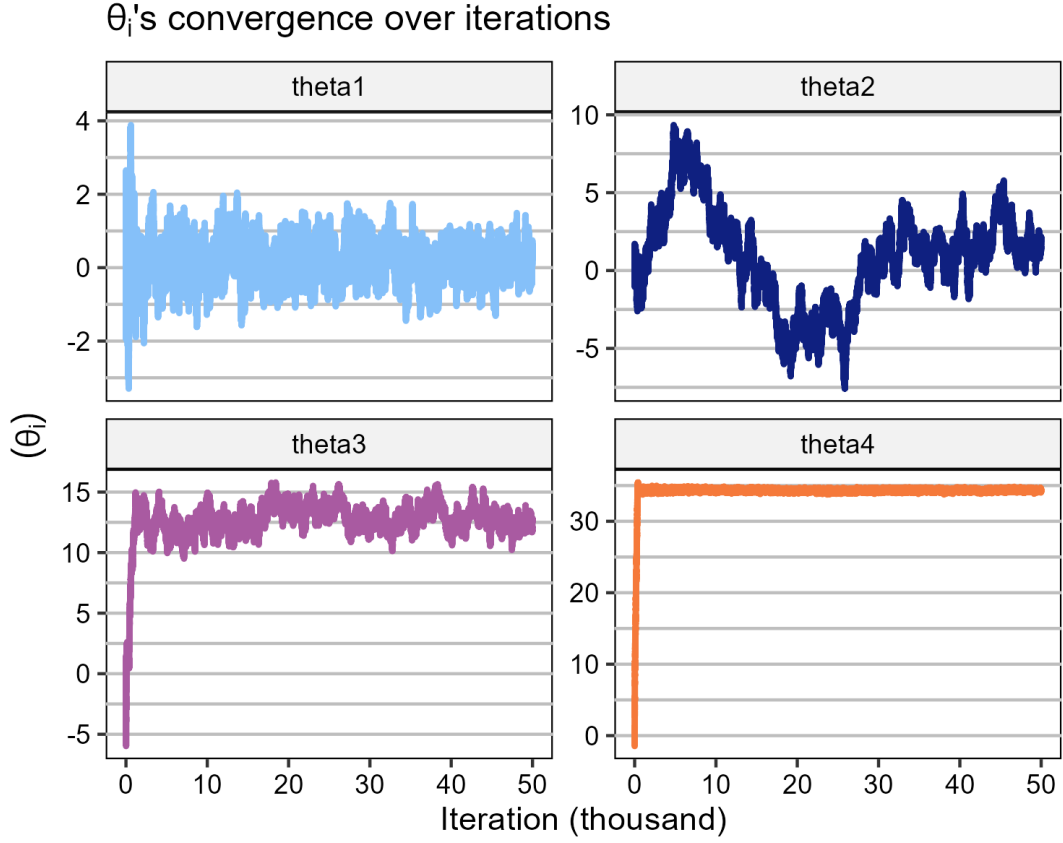


Figure 2: Evolution of parameters $\theta_1, \theta_2, \theta_3, \theta_4$ over iterations.

The estimated parameter vector is:

$$\hat{\theta} = (0.1422170, 0.8298934, 12.9221009, 34.3312660).$$

The corresponding value of the objective function is:

$$g(\hat{\theta}) = 0.02974863.$$

The algorithm required 30,195 iterations to converge. This low value of the loss function indicates an excellent fit of the model to the observed data.

Question 3

We want to compute:

$$\mathbb{E}[\max(X, Y)]$$

where $X, Y \sim \mathcal{N}(0, 1)$ are independent.

1. Gauss-Hermite Quadrature

To approximate the expectation, we use Gauss-Hermite quadrature. Since this method integrates functions of the form $e^{-x^2}f(x)$, we apply the transformation $x = \sqrt{2}z$ to match the standard normal density.

The expectation is then computed as a double sum over the quadrature nodes:

$$\mathbb{E}[\max(X, Y)] \approx \frac{1}{\pi} \sum_{i=1}^n \sum_{j=1}^n w_i w_j \max(x_i, x_j),$$

where $x_i = \sqrt{2}z_i$ and (z_i, w_i) are the Gauss-Hermite nodes and weights.

Using this method, we obtain:

$$\mathbb{E}[\max(X, Y)] \approx 0.5526643.$$

2. Monte Carlo Integration

We also approximate the expectation using Monte Carlo simulation. We draw a large number of independent samples (X_i, Y_i) from the standard normal distribution and compute:

$$\mathbb{E}[\max(X, Y)] \approx \frac{1}{N} \sum_{i=1}^N \max(X_i, Y_i).$$

Using $N = 10^6$ simulations, we obtain:

$$\mathbb{E}[\max(X, Y)] \approx 0.5642427.$$

Question 4

We approximate the following integrals using the trapezoid rule with $n = 3, 5, 10, 15, 20$ nodes:

$$\begin{aligned} \int_0^1 f_a(x) dx &= \int_0^1 x dx \\ \int_0^1 f_b(x) dx &= \int_0^1 x \sin(x) dx \\ \int_0^1 f_c(x) dx &= \int_0^1 \sqrt{1-x^2} dx \end{aligned}$$

The trapezoid rule approximates the integral by discretizing the interval into n equally spaced points and summing the areas of trapezoids formed by adjacent points.

Table 2: Numerical Integration - Trapezoid Rule

Nodes	fa	fb	fc
Analytical	0.5	0.301169	0.785398
3	0.5	0.330224	0.683013
5	0.5	0.308382	0.748927
10	0.5	0.302591	0.774546
15	0.5	0.301756	0.779798
20	0.5	0.301488	0.781854

Question 5

We approximate the derivatives of the following functions using centered finite differences with 2 and 4 points, for step sizes $h \in \{0.001, 0.005, 0.01, 0.05\}$:

$$\begin{aligned}
 f_a(x) &= x^2 & \text{at} & \quad x = 5 \\
 f_b(x) &= \log(x) & \text{at} & \quad x = 10 \\
 f_c(x) &= x \sin(x) & \text{at} & \quad x = 1
 \end{aligned}$$

Table 3: Numerical Differentiation

Function	Analytical	h = 0.001	h = 0.005	h = 0.01	h = 0.05
fa with 2 points	10	10	10	10	10
fa with 4 points	10	10	10	10	10
fb with 2 points	0.1	0.1	0.1	0.1	0.100001
fb with 4 points	0.1	0.1	0.1	0.1	0.1
fc with 2 points	1.381773	1.381773	1.381761	1.381722	1.380497
fc with 4 points	1.381773	1.381773	1.381773	1.381773	1.381772

For function (a), both methods are exact for all values of h , since the function is quadratic. For function (b), both methods perform well, with the 4-point method showing higher precision, especially for larger step sizes.

For function (c), the approximation error increases with h , particularly for the 2-point method. The 4-point method remains highly accurate across all step sizes, reflecting its higher-order convergence.

Overall, the results confirm that the 4-point centered difference provides a more accurate approximation, especially for nonlinear functions.